Compilers

Project overview

2nd Bachelor Computer Science
2025 – 2026

Kasper Engelen
kasper.engelen@uantwerpen.be

1 Introduction

In this document you will find practical information concerning the project that you will work on throughout the semester. The project will consist of implementing a compiler that compiles code written in (a subset of) the C language. The compiler will output LLVM IR code and MIPS and binary code. The project will be completed over the semester with weekly incremental assignments. During the project you will work in groups of two students. You can also work alone, but this will make the assignment obviously more challenging.

Concretely, you will construct a grammar and lexer specification, which is then turned into Python code using the ANTLR tool. The rest of the project will consist of developing custom code written in Python 3. This custom code will take the parse tree and turn it into an abstract syntax tree (AST). This AST will then be used to generate code in the LLVM IR language, which is in turn compiled into MIPS and binary code. At various stages of the project you will also have to develop utilities to provide insight into the internals of the compiler, such as visualising the AST tree.

2 Purpose of the project

Learning about compilers is an essential part of any computer science program. It combines multiple important aspects of our discipline: formal language theory and automata, data structures, algorithms, programming language theory, as well as programming, software architecture, assembly programming, low-level details, etc.

Concretely, there are a few technologies we use that could prove **useful later in your career**:

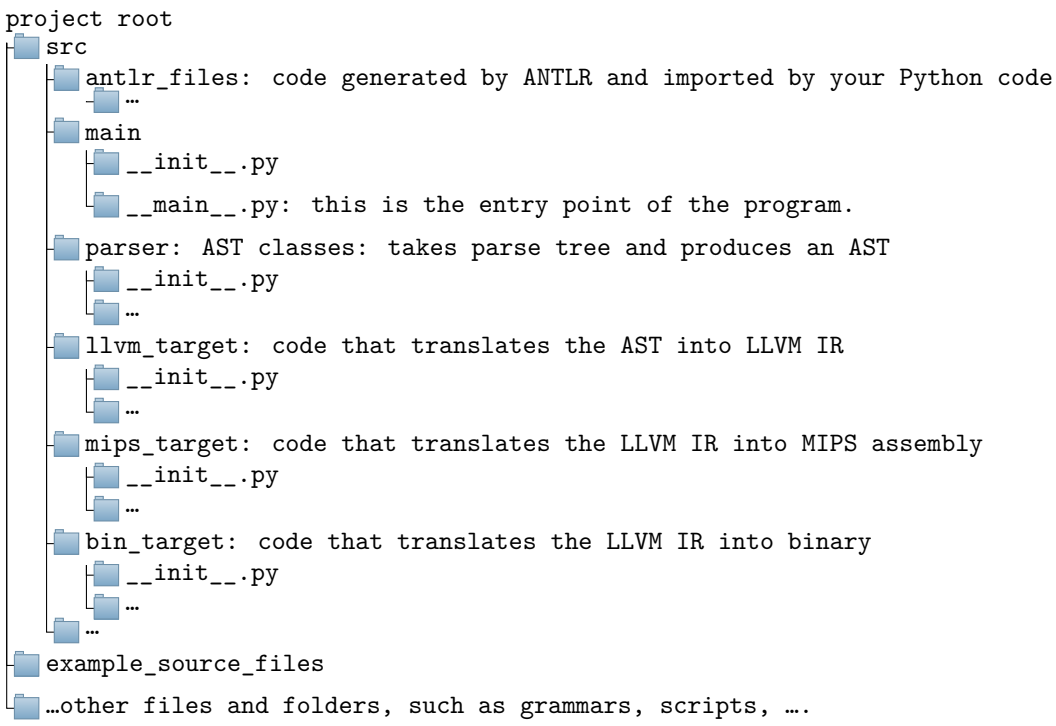
- **Parsers:** some companies use so-called "domain-specific languages" that will need to be maintained and require customized parsing and compilation,
- **Abstract-syntax trees:** useful for representing mathematical formulas and as well as data (e.g., XML, JSON). In some machine learning technologies they use something called 'autodiff' or 'symbolic differentiation'. ASTs are essential for this.
- **C, low-level programs, function pointers, etc.:** Many highly-optimised libraries (machine learning, scientific computing) make use of C. You will need to be able to deal with the quirks of the C language to use such libraries. Embedded software development is also often done in C.
- **LLVM:** there are interesting companies, both in Belgium and abroad, that require expertise in LLVM.

3 ANTLR

In this project we will use the ANTLR tool. On Blackboard you can find a PDF with additional information on ANTLR.

4 Project structure

In order to make sure that the assistant can read your code, give feedback, offer advice, and **grade your project**, it is important that a proper Python project structure is maintained. For example:



Invoking the compiler: In order to parse arguments, you can make use of the built-in `argparse` library. Your compiler can be accessed by issuing the following commands from the **project root directory**:

- Rendering the AST:

```
python -m src.main --input input_file.c --render_ast ast_output.dot
```

- Rendering the symbol table:

```
python -m src.main --input input_file.c --render_symb symb_output.dot
```

- Compile to LLVM:

```
python -m src.main --input input_file.c --target_llvm output_file.ll
```

- Compile to MIPS assembly:

```
python -m src.main --input input_file.c --target_mips output_file.mips
```

- Compile to a native binary:

```
python -m src.main --input input_file.c --target_bin output_file
```

Make sure your compiler is platform independent. In other words, take care to avoid absolute file paths in your source code.

5 Submitting your solution

In order to submit your solution, you will have to closely follow the steps below. Normally speaking, these instructions are final, but additional instructions might be issued via **Blackboard announcements**.

1. Hand in a `zip` file on Blackboard that contains the following:
   - a **README** file,
   - your **grammar**,
   - your Python **code**,
   - **C files** that demonstrate the functionality, both **mandatory** and **optional**,
   - a **test script that automatically runs your compiler** on the specified C files and, for each C file, produces two things:
     - the **AST** tree in Graphviz dot format,
     - some visualisation of the **symbol table**,
     - if applicable, the **LLVM IR** code,
     - if applicable, the **binary and MIPS** code.
   - a link to the **video** that you made.
2. Create a **video** in which you demonstrate your compiler:
   - Upload this video to YouTube or another online video platform. You may also use Mediasite. Make sure the link to your video works and **does not require an account/login.**
   - **Do not upload the video to Blackboard!**
   - The video should be **10 minutes long.**
3. The video should have the following "script":

(Step 1) Explicitly go over the **requirement list on Blackboard**, indicating for every item whether or not this is implemented.

(Step 2) Demonstrate that your compiler is capable of compiling to **LLVM IR**, **MIPS**, or **binary code**, depending on the assignment, by compiling the **test files on Blackboard**.

(Step 3) **Run** the compiled code in order to demonstrate that it works.

(Step 4) Show the **AST** and the **symbol table** for some interesting examples.

(Step 5) Make sure that your **optional functionality** is also demonstrated. The mandatory functionality is sufficient for a grade of 10/20. The optional functionality goes towards grades 11/20 – 20/20. Optional functionality will only be considered if the mandatory functionality is present.

6 Reference Compiler

If you want to compare certain properties (output, performance, etc.) of your compiler to a real-world compiler, the reference is the GNU C Compiler with options `-ansi` and `-pedantic`. The `-ansi` and `-pedantic` flags will tell GCC to **strictly** adhere to the C89 standard. Compiling a C source code file called `test.c` using GCC can be done as follows:

```
gcc -ansi -pedantic test.c
```

You can then study the behaviour of the compiler: warnings, errors, behaviour of the compiled code, etc.

When in doubt over the behavior of a piece of code (syntax error, semantical error, correct code, etc.), GCC 4.6.2 is the reference. Apart from that, you can consult the ISO and IEC standards, although only with regards to the basic requirements.

7 LLVM Reference

For a reference on the LLVM intermediate representation, it is best to consider the official documentation: `https://llvm.org/docs/LangRef.html`. On StackOverflow you may find more specific examples and explanations.

Alternatively, you can use Clang to output LLVM IR code for a given C file:

```
clang -S -emit-llvm test.c
```

Clang can also be accessed using an online interactive tool called "Compiler Explorer": `https://godbolt.org/`. In order to get it to output LLVM IR for the C language, you can use the following options:

- Language: C
- Compiler: `x86-64 clang 16.0.0`
- Compiler options: `-S -emit-llvm`

8 MIPS Reference

There are a number of useful sources regarding the syntax and semantics of MIPS:

- the **CSA course** of the 1st bachelor year,
- various MIPS **reference sheets**,
- **Stackoverflow** for specific questions and problems,
- the **auto-complete** feature of the MARS simulator,
- a list of **system calls** and their respective arguments: Link,
- *Computer Organization and Design, MIPS Edition* by David Patterson and John Hennessy.

Clang for MIPS can be accessed using an online interactive tool called "Compiler Explorer": `https://godbolt.org/`. In order to get it to output MIPS assembly for the C language, you can use the following options:

- Language: C
- Compiler: `mips clang 16.0.0` (GCC works also, but does not use the proper register names)
- Compiler options: leave empty

Note: The output of Clang does not fully correspond with the MIPS code your compiler needs to generate:

- The code generated by "real" compilers may not always work on your emulator, since emulators do not always provide the full functionality of the MIPS instruction set. In the case that happens, you'll have to manually correct the MIPS code generated by Clang to make it work in MARS or SPIM.
- Also note that Clang does not generate an `exit` syscall at the end of the `main` function, so you'll have to add this yourself.
- You'll manually have to implement `printf` and `scanf`. Clang will sometimes just do `jal printf`, which is not sufficient for this project.

9 Test files

On Blackboard you will find examples of C-code. You can use these to test your compiler and to demonstrate during the evaluation video that your compiler works as intended. Note, however, that these files are **not exhaustive**. They are provided to help you find bugs in your compiler.

It is up to you to properly test the compiler, and you are expected to implement all required functionalities, as well as the optional functionalities that you indicated. If certain functionalities are not properly implemented, points will be deducted.

The test files are just a collection of code snippets. It is up to you to properly use them to detect bugs in your compiler. Run the test files using GCC in order to figure out how your compiler needs to handle the test files.

10 Code generation

Please pay attention to the following:

- All features should be implemented for both LLVM IR and assembly by the end of this project. Features that are only implemented for one of the two targets do not count.
- You should use a library to generate LLVM IR, such as llvmlite (Example). This will allow you to build your LLVM code in an object-oriented manner.
- Use existing tooling to compile from LLVM IR to MIPS and native binary. Either use a library or a command-line tool for this. No GUI-based or online tools will be allowed.

11 Deadlines

The following deadlines are strict:

- By **Friday 21 February 2025**, you should sign up for one of the groups **on Blackboard**.
- By **Friday 21 March 2025**, you should be able to demonstrate that your compiler is capable of compiling a small subset of C to the intermediary LLVM language. This will be defined in project assignments 1 – 3. **You only need to submit your code as a zip-file via Blackboard.**
- By **Friday 9 May 2025**, you should be able to demonstrate that your compiler is capable of compiling C to the intermediary LLVM language (all assignments). Indicate in the README file which optionals you chose to implement. The semantic analysis should be complete now.
- By **Friday 30 May 2025**, output to MIPS and binary should be working. LLVM IR functionality will **NOT** be graded at this stage! You are allowed so fix bugs in the LLVM IR, however, if that is necessary for correct MIPS and binary output.

No solutions will be accepted via e-mail; only timely submissions posted on BlackBoard will be accepted and assessed.

12 Grading

The project counts for 60% of the entire course (=12/20). The LLVM IR part counts for 90% of the project ($\approx$ 10,8/12), the binary and MIPS part counts for 10% of the project. There is no re-take (tweede zit) for the project. In order to give everyone a fair grade, we may deviate from this scoring method if we see problems with quality, if there are difficulties with collaborating between group members, etc.

If you implement **all mandatory functionality**, then you get 50% of the points for that part of the project. The amount of implemented optional functionality is then used to determine a grade between 50% and 100% for that part of the project.

We first take a look at how many mandatory features of LLVM IR you implemented, by counting the number of implemented features and dividing by the total amount of features, so that the total grade is:

$$\frac{\text{number of implemented mandatory features}}{\text{total number of mandatory features}} \times 0.5 \times 0.9$$

If you implemented all mandatory LLVM IR features, we also count the number of optional features in a similar way, and then the following is added to your grade:

$$\frac{\text{number of implemented optional features}}{\text{total number of optional features}} \times 0.5 \times 0.9.$$

If all mandatory LLVM features are implemented, we will look at the MIPS and binary output. If the MIPS and binary output is complete, your score will be increase with an additional 10/100 points.

Note that you can also come up with your own optional features, which will also count towards a higher grade.

Another thing to consider: just doing all the mandatory LLVM IR features is not enough to pass the project, since then you will only get 45% on the project.